

DAY 1



INTRODUCTION TO LANGUAGE AI



What Is Language AI?

Language AI (Language Artificial Intelligence) is a specialized branch of artificial intelligence dedicated to creating technologies that can understand, process, and generate human language.

Because of the widespread success of machine learning in solving complex text tasks, the term Language AI is often used interchangeably with Natural Language Processing (NLP). While high-profile Large Language Models (LLMs) like ChatGPT have brought massive attention to the field, Language AI spans a much broader ecosystem. It includes:

- Generative Models (Decoder-only models like GPT) designed to generate human-like text.
- Representation Models (Encoder-only models like BERT) designed to analyze, classify, and extract meaning from text.
- Embeddings & Search Systems that allow computers to match and retrieve information based on conceptual meaning rather than exact word matching



- We use the term Language AI to encompass technologies that technically might not be LLMs but still have a significant impact on the field, like how retrieval systems can give LLMs superpowers.
- Throughout this course, we want to focus on the models that have had a major role in shaping the field of Language AI. This means exploring more than just LLMs in isolation. That, however, brings us to the question: what are large language models? To begin answering this question in this chapter, let's first explore the history of Language AI.

How Does Language AI Work?

Computers do not natively understand human words; they only process numbers (zeros and ones). Language AI works by transforming unstructured human language into numerical representations, processing those numbers through neural networks, and converting the outputs back into text or structured predictions.



Here is a step-by-step breakdown of how modern Language AI systems operate under the hood:

1. Tokenization (Breaking Down the Text)

Before an AI model can analyze a text prompt, the text passes through a tokenizer. Tokenization splits unstructured text into smaller chunks called tokens, which can be individual words, subwords, or characters.

- Example: The sentence "That is a cute dog" is split on whitespace into the tokens ["That", "is", "a", "cute", "dog"]. Subword tokenization allows models to handle unknown or complex words by breaking them into smaller meaningful fragments.

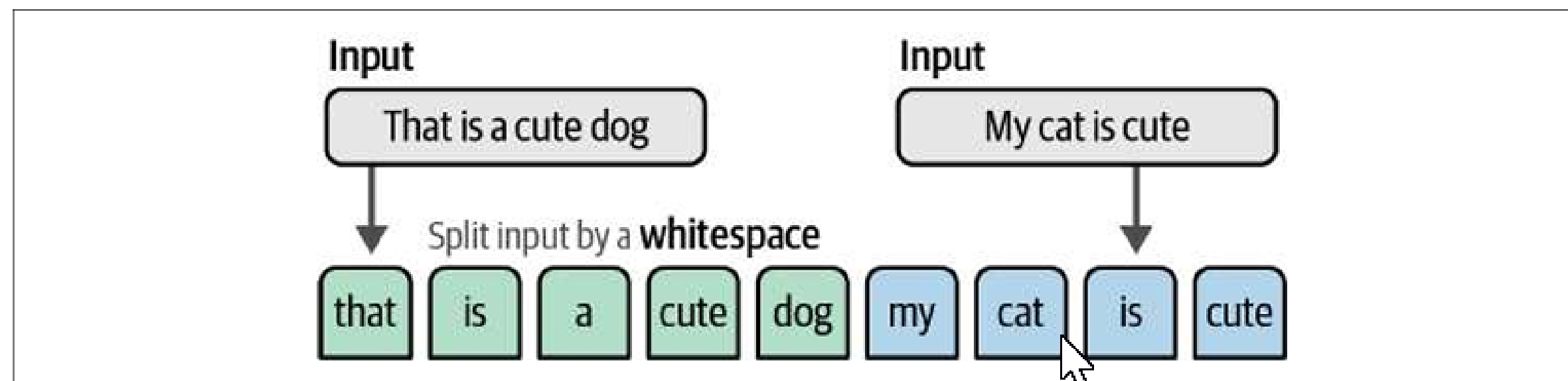


Figure 1-3. Each sentence is split into words (tokens) by splitting on a whitespace.



2. Embeddings (Converting Words into Numbers)

Once text is tokenized, each token is mapped to a dense numerical vector called an embedding. Embeddings place words into a high-dimensional mathematical space where words with similar meanings are located closer together.

- **Static vs. Contextual Embeddings:** Early embedding algorithms like word2vec assigned a single static vector to each word. Modern Language AI uses contextualized embeddings, where the vector representation of a word dynamically changes depending on the surrounding sentence. For instance, the word "bank" receives a different numerical vector in "river bank" versus "investment bank"



3. The Transformer Architecture & Self-Attention

Most modern Language AI systems rely on the Transformer architecture. The core innovation of the Transformer is the self-attention mechanism, which allows the model to evaluate how every word in a sequence relates to every other word.

- **Example:** Consider the prompt: "The dog chased the squirrel because it...". To predict what word comes next, the model must determine what "it" refers to. The self-attention mechanism calculates relevance scores across the tokens and heavily connects "it" to "squirrel" (or "dog"), incorporating that context into its final decision.



4. Autoregressive Prediction & Generation

Generative Language AI models are completion machines. They operate autoregressively, meaning they generate output text one token at a time. At each step, the model computes a probability distribution over its entire vocabulary to pick the most likely next token, appends that token to its context, and repeats the loop.

- **Example:** If given the input prompt "So long and thanks for", the model calculates probabilities for possible next words (e.g., 44% probability for "all", 33% for "the"), selects "all", adds it to the prompt, and then predicts the following token ("the").

A Recent History of Language AI



Language, however, is a tricky concept for computers. Text is unstructured in nature and loses its meaning when represented by zeros and ones (individual characters). As a result, throughout the history of Language AI, there has been a large focus on representing language in a structured manner so that it can more easily be used by computers. Examples of these Language AI tasks are provided in Figure 1-2.

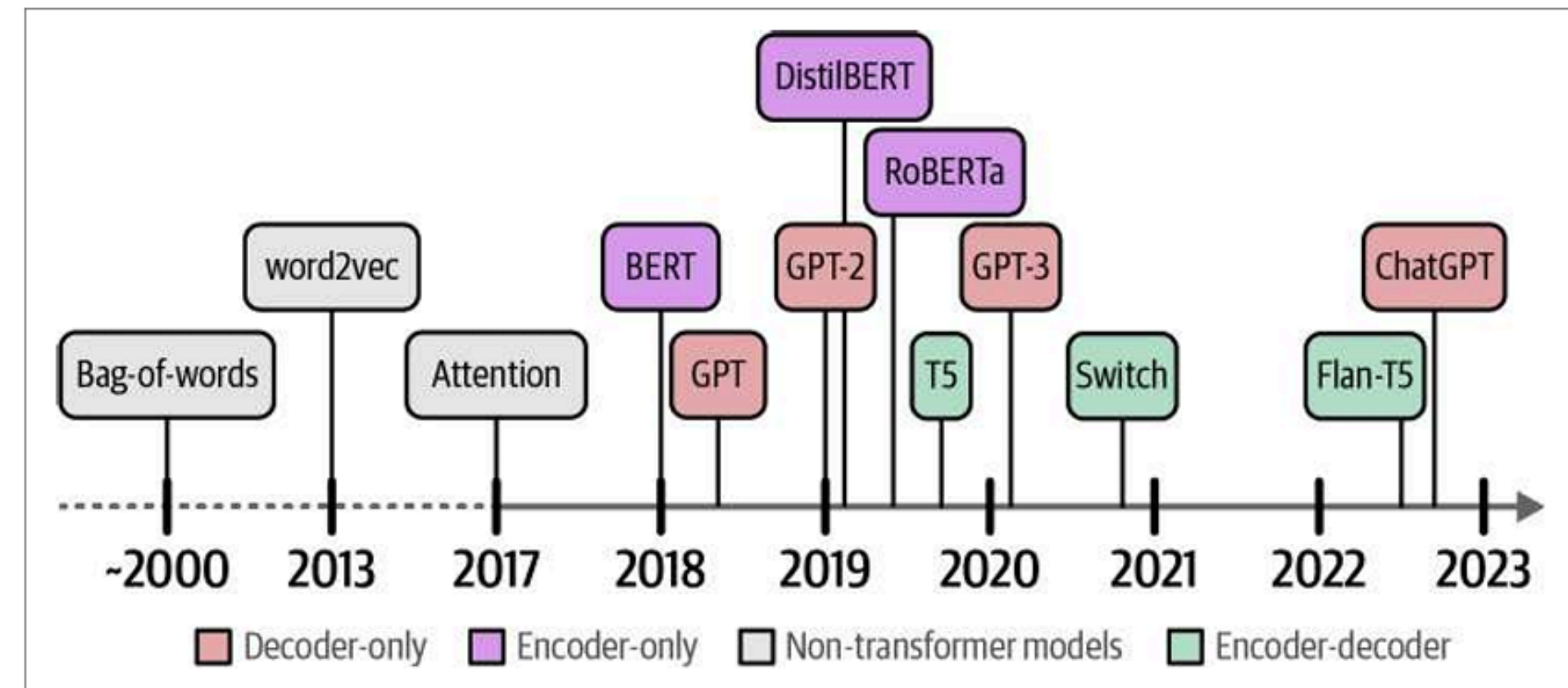


Figure 1-1. A peek into the history of Language AI.

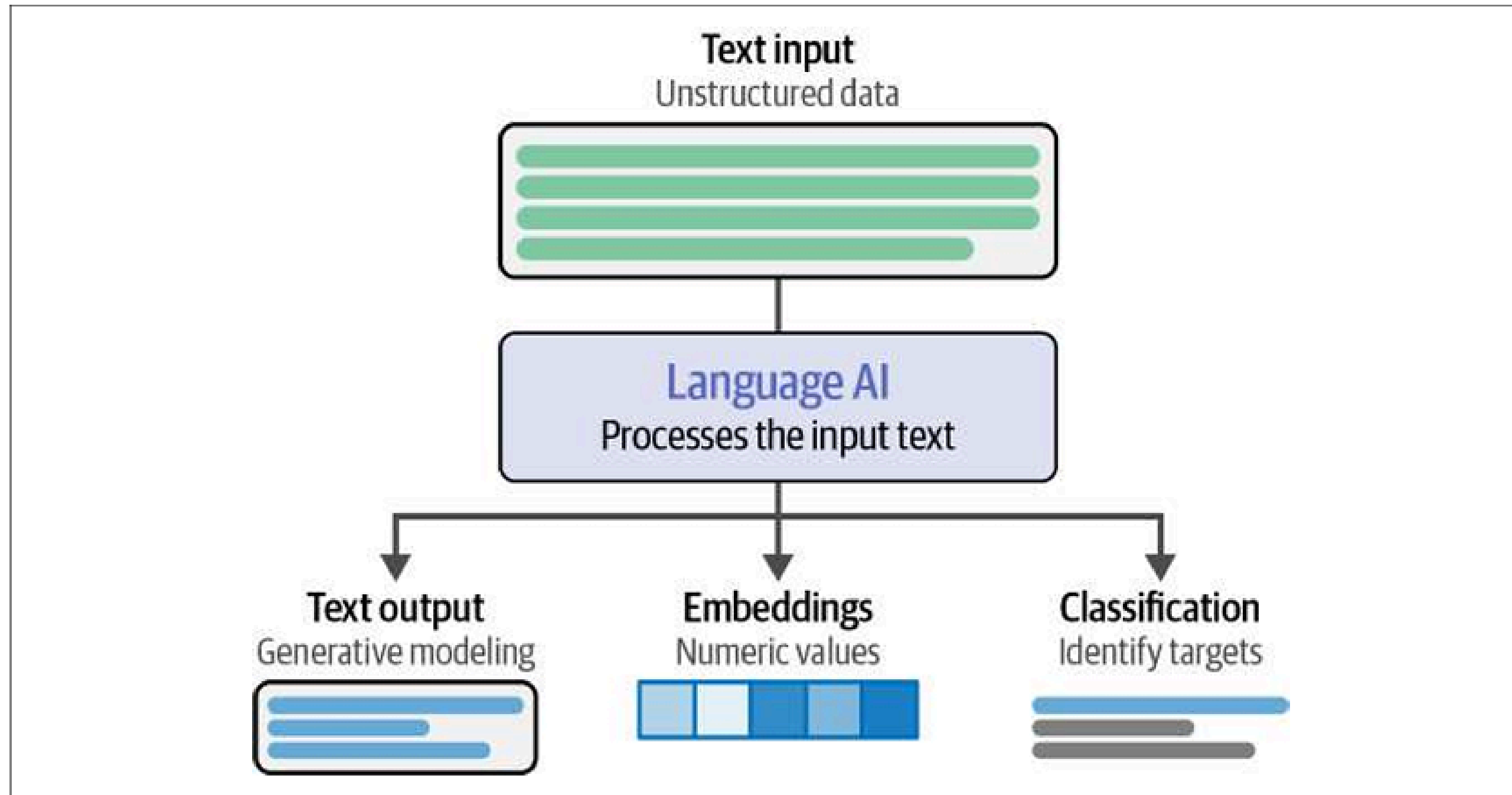


Figure 1-2. Language AI is capable of many tasks by processing textual input.

Representing Language as a Bag-of-Words



Our history of Language AI starts with a technique called bag-of-words, a method for representing unstructured text. It was first mentioned around the 1950s but became popular around the 2000s.

Bag-of-words works as follows:

let's assume that we have two sentences for which we want to create numerical representations. The first step of the bag-of-words model is tokenization, the process of splitting up the sentences into individual words or subwords (tokens), as illustrated in Figure 1-3.

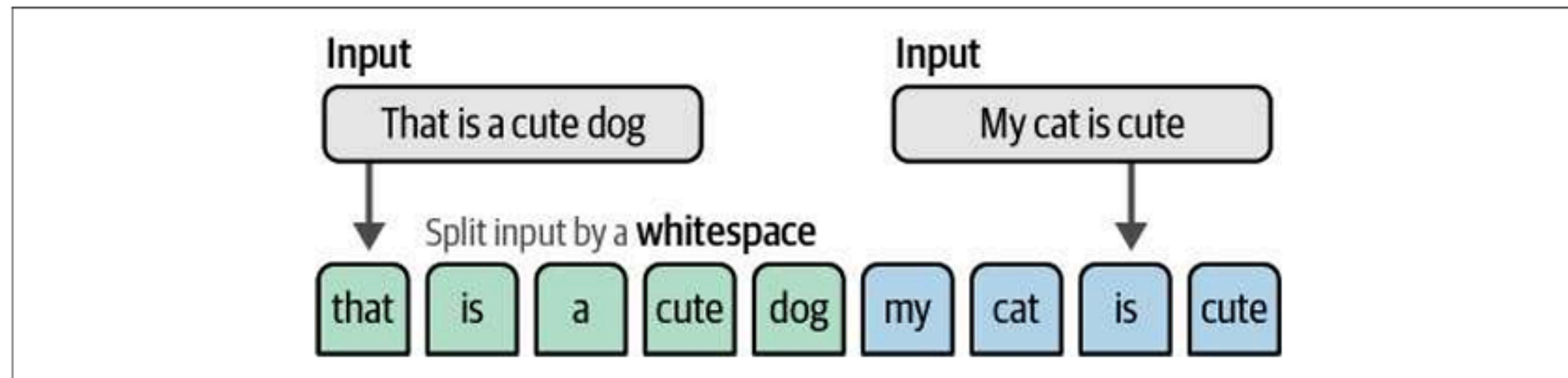


Figure 1-3. Each sentence is split into words (tokens) by splitting on a whitespace.

The most common method for tokenization is by splitting on a **whitespace** to create individual words. However, this has its disadvantages as some languages, like **Man-darin**, do not have whitespaces around individual words. In the next chapter, we will go in depth about tokenization and how that technique influences language models. As illustrated in Figure 1-4, after tokenization, we combine all unique words from each sentence to create a vocabulary that we can use to represent the sentences.

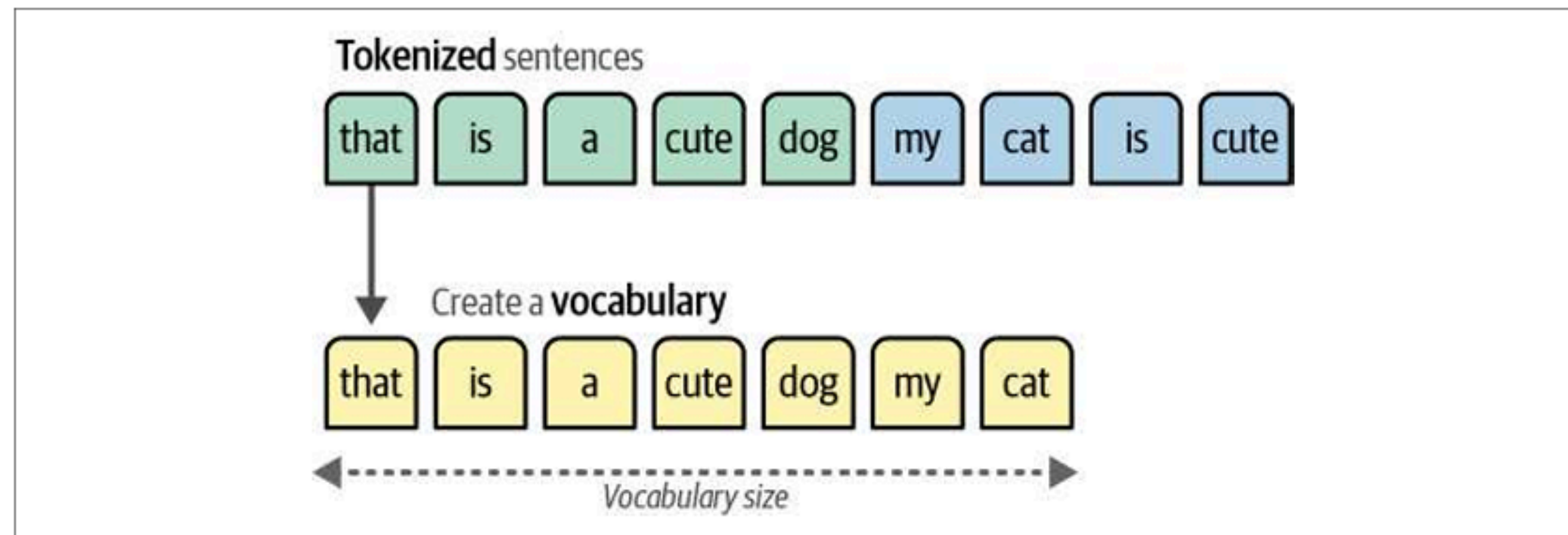


Figure 1-4. A vocabulary is created by retaining all unique words across both sentences.



Using our vocabulary, we simply count how often a word in each sentence appears, quite literally creating a bag of words. As a result, a bag-of-words model aims to create representations of text in the form of numbers, also called vectors or vector representations, observed in Figure 1-5. Throughout the course, we refer to these kinds of models as representation models

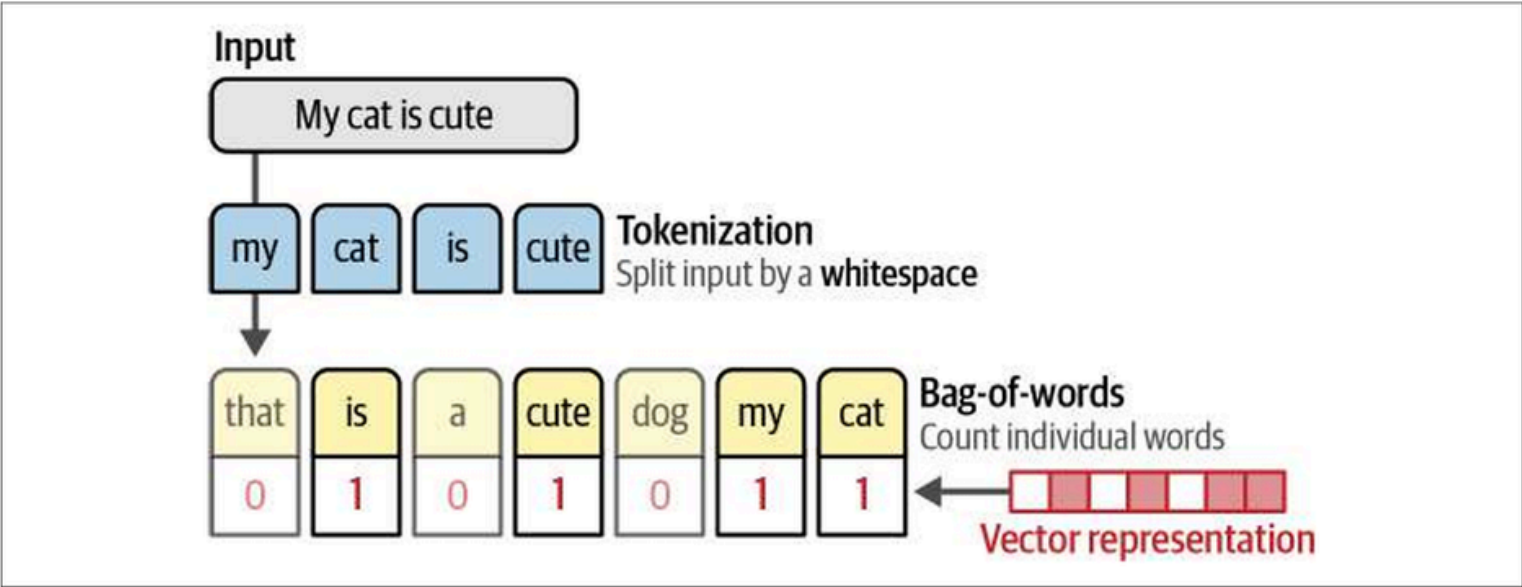
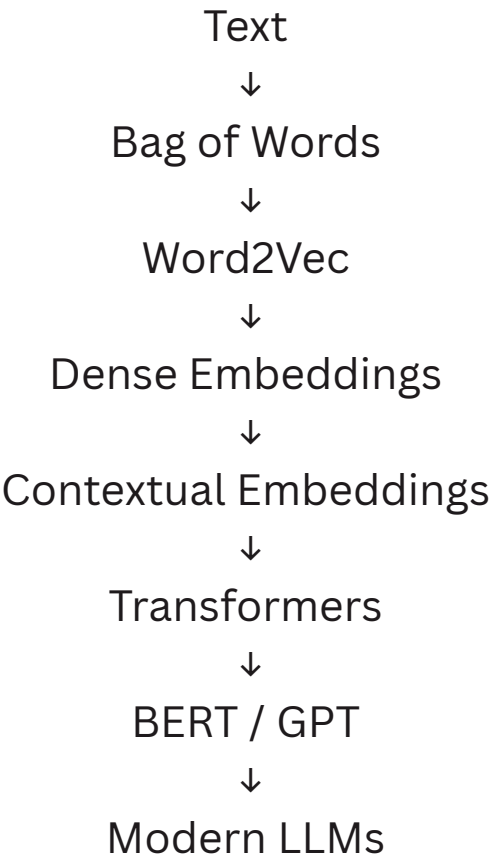
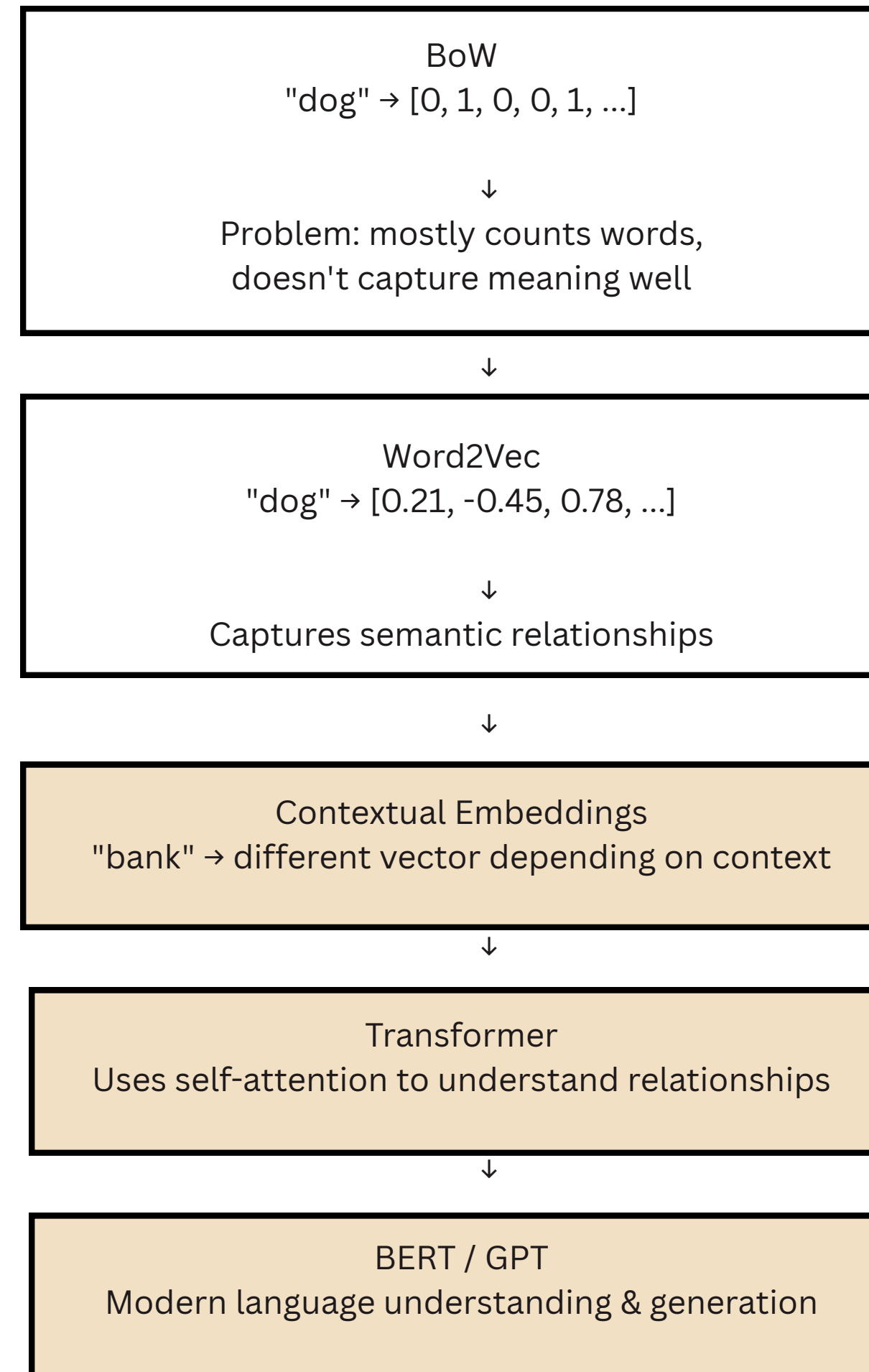
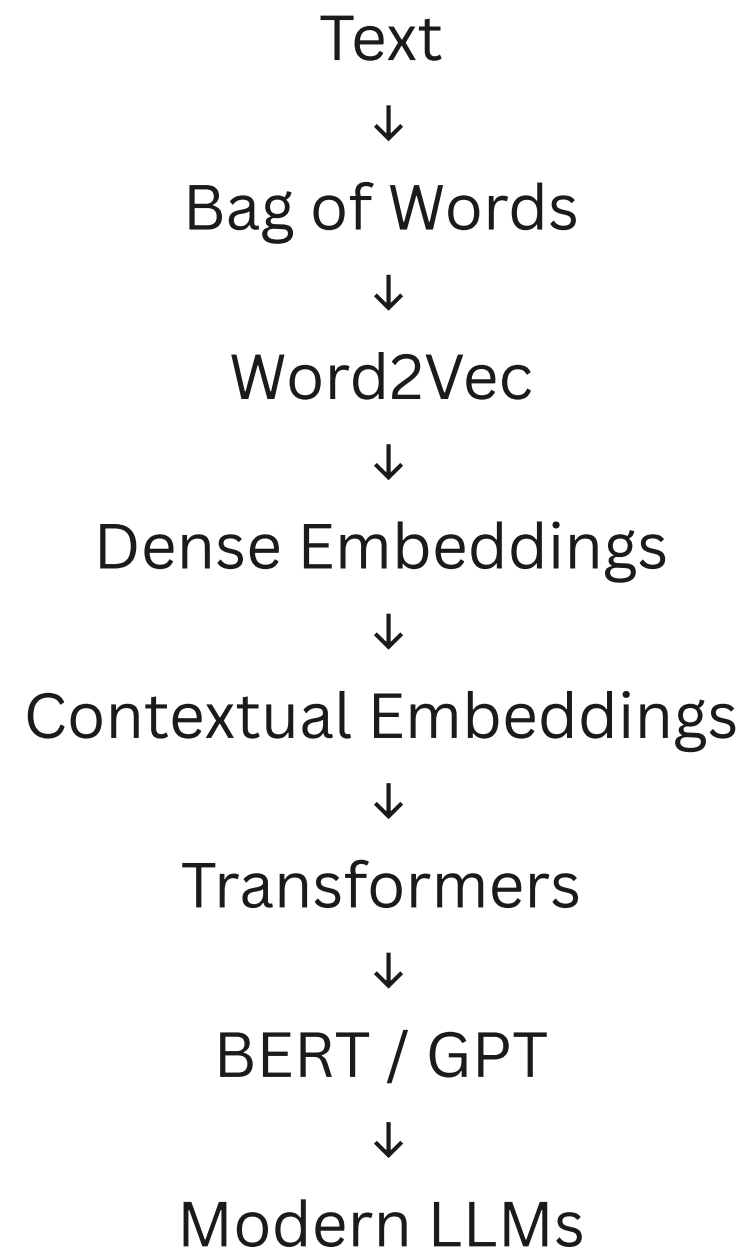


Figure 1-5. A bag-of-words is created by counting individual words. These values are referred to as vector representations.



		the	cat	is	on	table	dog	in	park	
1										
2	"The cat is on the table."	2	1	1	1	1	0	0	0	
3	"The dog is in the park."	2	0	1	0	0	1	1	1	
4	"The table is in the park."	2	0	1	0	1	0	1	1	



Better Representations with Dense Vector Embeddings

- Bag-of-words, although an elegant approach, has a flaw. It considers language to be nothing more than an almost literal bag of words and ignores the semantic nature, or meaning, of text.
- Released in 2013, word2vec was one of the first successful attempts at capturing the meaning of text in embeddings. Embeddings are vector representations of data that attempt to capture its meaning. To do so, word2vec learns semantic representations of words by training on vast amounts of textual data, like the entirety of Wikipedia.
- To generate these semantic representations, word2vec leverages neural networks. These networks consist of interconnected layers of nodes that process information. As illustrated in Figure 1-6, neural networks can have many layers where each connection has a certain weight depending on the input. These weights are often referred to as the parameters of the model.

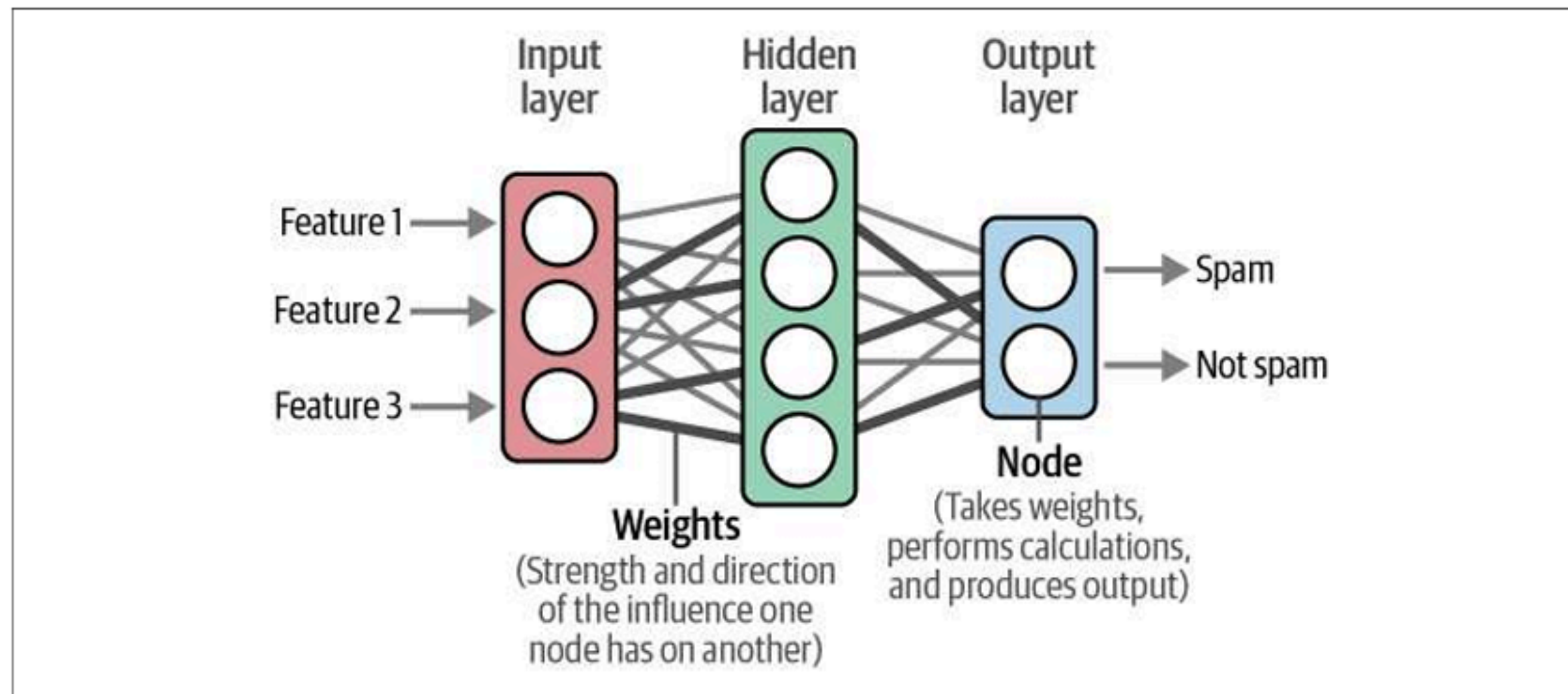


Figure 1-6. A neural network consists of interconnected layers of nodes where each connection is a linear equation.

- Using these neural networks, word2vec generates word embeddings by looking at which other words they tend to appear next to in a given sentence. We start by assigning every word in our vocabulary with a vector embedding, say of 50 values for each word initialized with random values. Then in every training step, as illustrated in Figure 1-7, we take pairs of words from the training data and a model attempts to predict whether or not they are likely to be neighbors in a sentence

- During this training process, word2vec learns the relationship between words and distills that information into the embedding. If the two words tend to have the same neighbors, their embeddings will be closer to one another and vice versa. we will look closer at word2vec's training procedure.

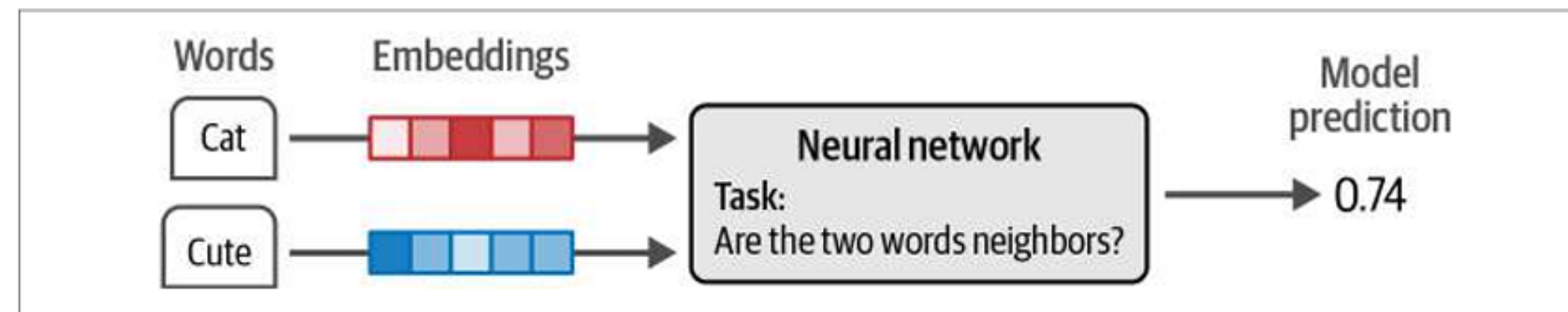


Figure 1-7. A neural network is trained to predict if two words are neighbors. During this process, the embeddings are updated to be in line with the ground truth.

- The resulting embeddings capture the meaning of words but what exactly does that mean? To illustrate this phenomenon, let's somewhat oversimplify and imagine we have embeddings of several words, namely "apple" and "baby." Embeddings attempt to capture meaning by representing the properties of words. For instance, the word "baby" might score high on the properties "newborn" and "human" while the word "apple" scores low on these properties.

- As illustrated in Figure 1-8, embeddings can have many properties to represent the meaning of a word. Since the size of embeddings is fixed, their properties are chosen to create a mental representation of the word.

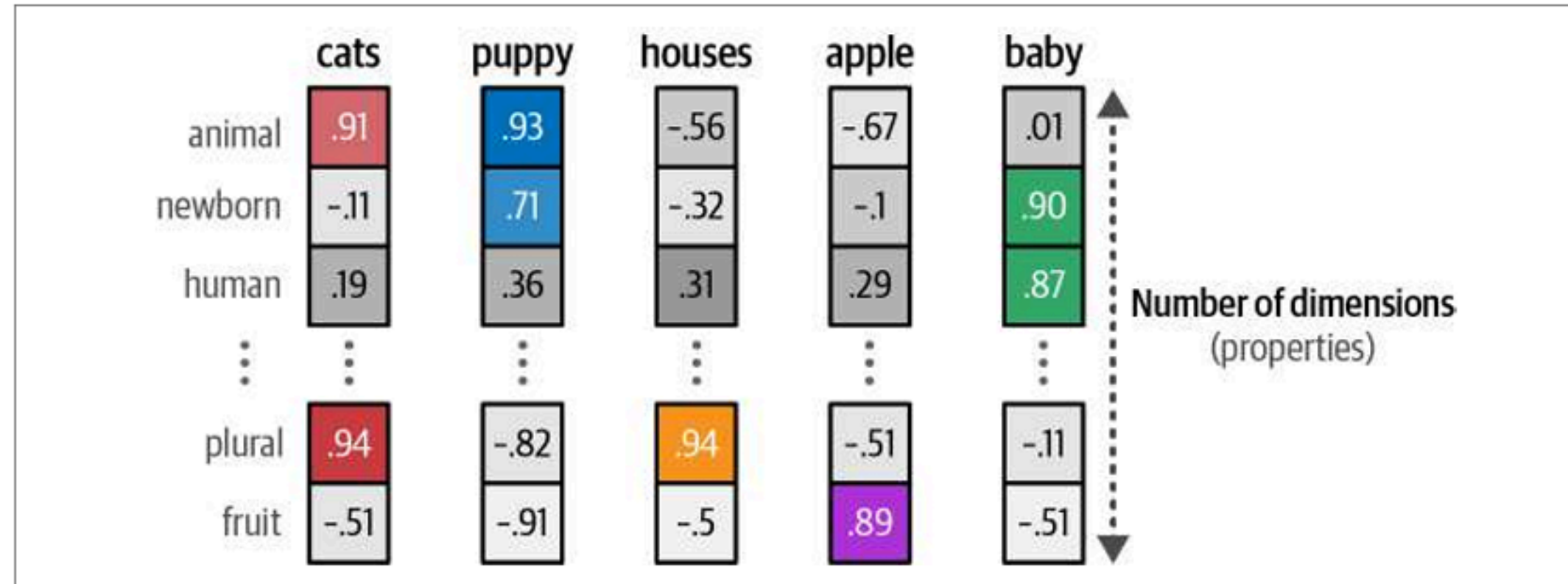


Figure 1-8. The values of embeddings represent properties that are used to represent words. We may oversimplify by imagining that dimensions represent concepts (which they don't), but it helps express the idea.

- In practice, these properties are often quite obscure and seldom relate to a single entity or humanly identifiable concept. However, together, these properties make sense to a computer and serve as a good way to translate human language into computer language.

- Embeddings are tremendously helpful as they allow us to measure the semantic similarity between two words. Using various distance metrics, we can judge how close one word is to another. As illustrated in Figure 1-9, if we were to compress these embeddings into a two-dimensional representation, you would notice that words with similar meaning tend to be closer. we will explore how to compress these embeddings into n-dimensional space in upcoming classes.

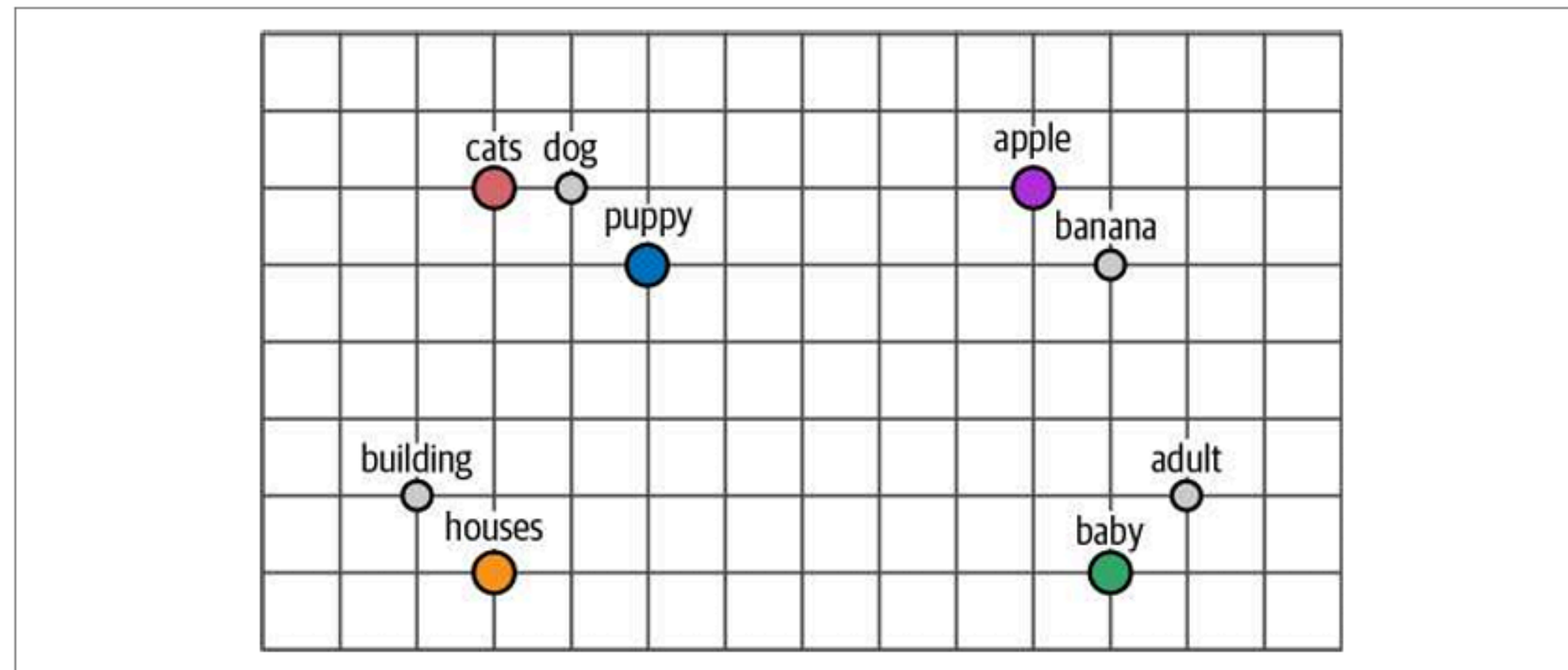


Figure 1-9. Embeddings of words that are similar will be close to each other in dimensional space.

COURSE 1

LARGE LANGUAGE MODELS (LLMs) & AGENTIC AI

